NUMCOMPUTE • TEAM PROJECT PLAN

三人团队分工方案

针对 Assignment 2.1 的三套可执行分工方案：角色划分、时间表、依赖关系、优缺点与最终推荐。

团队规模：3 人 | 启动：2026-10-05 | Due: Mon, 26 Oct 2026 9:00 | 核心目标：并行开发 + 稳定集成

最推荐

方案一

按功能领域分工 + 后期统一集成

三个人分别负责数据预处理、算法统计、ML/集成，前两周高并行开发，第三周集中做测试、Benchmark、Demo、报告与视频。

内部最终截止

10 月 25 日 22:00

建议不要把 10 月 26 日 09:00 当作真正的开发截止时间。

你们真正要解决的问题

TEAM STRATEGY

① 并行度

尽量让三个人同时推进，不因为某个模块未完成而整体停工。

② 接口一致

提前统一 `fit`、`transform`、`axis`、NaN、异常类型和 shape 语义。

③ 留出集成期

代码不是最后一天拼起来；至少留 7-10 天做测试、Demo、Benchmark 和文档。

建议的四个内部节点： 10/13 Core Code Deadline → 10/16 Integration Deadline → 10/22 Documentation Deadline → 10/25 Internal Final Deadline

PLAN 01 • RECOMMENDED

方案一：按功能领域分工 + 后期统一集成

最推荐

最适合三人团队：模块天然分区、前期依赖少、并行度高；后期再由 Integration Owner 统一 API、测试和 Demo。

成员 A

Data / Preprocessing

- `io.py`
- `preprocessing.py`
- `utils.py`

CSV、缺失值、Scaler、OneHot、输入验证、NaN、shape。

成员 B

Algorithms / Statistics

- `sort_search.py`
- `rank.py`
- `stats.py`

top-k、quickselect、ties、percentile、quantile、axis。

成员 C

ML / Integration

- `metrics.py`
- `optim.py`
- `pipeline.py`
- `benchmarking.py`

同时担任 Integration Owner，负责最终串联。

工作量为什么仍然较平衡？ C 文件更多但 metrics / optim 较独立；B 的 quickselect、ties、NaN/axis 细节更耗时；A 的 preprocessing 需要大量边界处理。

成员 A • 交付重点

- CSV delimiter / dtype / missing values
- StandardScaler / MinMaxScaler
- OneHotEncoder + optional Imputer
- utils：logsumexp、batching、distance
- 测试：zero variance、NaN、1D/2D、shape mismatch

成员 B • 交付重点

- stable / multi-key sort
- top-k、quickselect、binary search
- average / dense / ordinal rank
- histogram、quantile、descriptive stats
- 测试：duplicates、all-equal、k 极值、non-contiguous

成员 C • 交付重点

- accuracy / precision / recall / F1 / MSE
- confusion matrix
- gradient / Jacobian
- Pipeline 串联 Transformer
- benchmark harness + 最终 integration

主要优点

- 并行度最高，前期几乎不互相等待。
- 贡献边界最清晰，Teamwork documentation 好写。
- 模块逻辑自然，适合约三周周期。

主要风险

- 可能写成“三个风格不同的库”。
- 10/5-10/6 必须统一 API、错误处理和 docstring。
- C 的最终整合责任较重。

PLAN 01 • TIMELINE

方案一时间表与依赖关系

2026-10-05 → 2026-10-25

日期	成员 A	成员 B	成员 C
10/5-10/6	**全员统一 API / 规范。**确认 1D/2D、axis、NaN、异常类型、fit 返回值、docstring、命名和 shape；**10/6 晚冻结第一版 API。**		
10/7-10/8	`io.py`	stable sort + binary search	`metrics.py`
10/9-10/10	StandardScaler + MinMaxScaler	top-k + quickselect	`optim.py` + Pipeline skeleton
10/11-10/13	OneHot + utils + ⩾7 tests	rank + stats + ⩾7 tests	pipeline + benchmarking + ⩾7 tests
10/13 晚	**Core Code Deadline：**所有核心模块可运行，总测试建议 21-25+；之后原则上 API Freeze。		
10/14-10/16	测试 B	测试 C	测试 A
10/16 晚	**Integration Deadline：**交叉测试完成，主要兼容问题修复。		
10/17-10/19	preprocessing benchmark/example	sorting/stats benchmark/example	主导 benchmark + 合并 quickstart.ipynb
10/20-10/22	README：安装 + preprocessing	README：algorithms + stats	README：pipeline + benchmark；整合 report
10/23-10/24	完成 ⩽5 分钟 Demo Video，检查演示流程、输出与讲解。		
10/25	**只修 Bug，不加功能。**pytest、fresh install、notebook 全运行、README 命令、Repo、PDF Report、Video、文件名。		
10/25 22:00	**Internal Final Deadline。**		

交叉测试

A 测 B →

B 测 C →

C 测 A

避免“自己写、自己测、自己觉得没问题”。

关键依赖

A: preprocessing →

C: pipeline

除此以外，sort/rank/stats/metrics/optim 基本可并行。

A/B/C modules → Benchmark → Quickstart → README / Report → Video

PLAN 02

方案二：架构 / 算法 / 质量三条线

一致性优先

更接近真实软件团队的角色划分：一个人负责框架，一个人负责算法，一个人负责 QA 与 Integration。

成员 A

Framework / Data

- `io.py`
- `preprocessing.py`
- `pipeline.py`
- packaging

成员 B

Algorithms / Numerical

- `sort_search.py`
- `rank.py`
- `stats.py`
- `optim.py`

成员 C

QA / Integration

- `metrics.py`
- `utils.py`
- `benchmarking.py`
- tests / quickstart / README coordinator

时间表

10/5-10/6	**A 主导 API；B 检查算法接口；C 建 tests / benchmark / demo 基础结构。**
10/7-10/12	A：10/8 io，10/10 preprocessing，10/12 pipeline。 B：10/8 sort/search，10/10 rank/stats，10/12 optim。 C：10/8 metrics，10/9 utils，10/10 test framework，10/12 benchmark framework。
10/13-10/16	C 主导 integration；A/B 按 C 的测试报告修 bug。目标：10/16 主要 tests 全部通过。
10/17-10/19	C 主导 benchmark 与 quickstart；A/B 提供例子、数据和各模块说明。
10/20-10/22	README、4 页 report、performance table。
10/23-10/24	录制与剪辑 Demo Video。
10/25	Final QA；只修 bug，不新增功能。

依赖关系

A: Framework **B: Algorithms** → **C: QA / Integration**

优点

- 项目一致性通常最好。
- 有明确 QA Owner，shape、异常、docstring、edge cases 容易统一。
- Repo 最终往往更整洁。
- 适合团队里有一位特别细心、擅长 Debug 和整理的人。

缺点

- B 的算法任务很重。
- C 前期可能偏空，后期突然很忙。
- C 的很多工作依赖 A/B 完成，因此并行性弱于方案一。
- 如果 A/B 延迟，integration 会被连锁拖延。

PLAN 03

方案三：全员按阶段一起做

共同所有权

不固定“某个文件归谁”，而是按阶段推进。每个功能由一人主写、一人测试、一人 Review，并在下一功能轮换角色。

日期	全员目标
10/5-10/6	API + architecture
10/7-10/9	io + preprocessing
10/10-10/13	sort / search / rank / stats
10/14-10/15	metrics + optim
10/16-10/17	pipeline
10/18-10/19	tests
10/20	benchmarks
10/21	quickstart
10/22	README
10/23	report
10/24	video
10/25	final QA

轮换方式示例

quickselect：A 主写算法 → B 写 tests → C 检查复杂度、docstring、edge cases。

StandardScaler：B 主写 → C 测试 → A Review。

下一模块继续轮换。

优点

- 代码一致性很高。
- 三个人几乎都理解整个项目，Bus factor 最低。
- 最终 Presentation 时每个人都能讲整体。
- Review 密度高。

缺点

- 并行能力最差。
- 容易三个人同时碰一个文件，产生 Git conflict。
- 讨论成本高，同一小时可能只推进一个功能。
- 贡献归属难写，容易变成 “Everyone contributed to everything”。

适用条件：如果你们三个人经常能同时见面、技术水平接近、愿意高频 Pair/Group Programming，这个方案会比较舒服；否则不建议作为主方案。

COMPARISON

三套方案横向比较

DECISION MATRIX

维度	方案一 功能领域分工	方案二 架构/算法/质量	方案三 全员按阶段
并行效率	★★★★★	★★★★☆	★★☆☆☆
工作量平衡	★★★★★	★★★☆☆	★★★★☆
代码一致性	★★★★☆	★★★★★	★★★★★
Integration 风险	★★★★☆	★★★★★	★★★★★
Git conflict 风险	★★★★★	★★★★★	★★☆☆☆
Contribution 清晰	★★★★★	★★★★★	★★☆☆☆
适合 3 人团队	★★★★★	★★★★☆	★★★☆☆
推荐度	★★★★★	★★★★☆	★★★☆☆

选择方案一，如果

你们想最大化开发效率

三个人希望前两周尽量独立推进，同时又希望贡献记录非常清楚。

选择方案二，如果

团队里有很强的 QA / 整理型成员

你们更看重最终一致性，且能接受后半程由一人承担较多集成工作。

选择方案三，如果

三个人经常能同时线下/在线协作

技术水平接近，并愿意持续进行 Pair / Group Programming；否则其并行效率通常不够理想。

FINAL RECOMMENDATION

最终建议：方案一 + 方案二的 Integration Owner

建议直接采用

保留方案一的高并行分工，同时借用方案二的明确集成责任：成员 C 负责最终 API 串联，但 QA 仍由三个人交叉承担。

A
Data
`io` • `preprocessing` • `utils`

B
Algorithms
`sort_search` • `rank` • `stats`

C
ML + Integration
`metrics` • `optim` • `pipeline` • `benchmarking`

A: Data | **B: Algorithms** | **C: ML/Numerical** → **C: Integration** → **Cross-review** → **Demo / Report**

关键纪律：可以分别开发，但不允许只测试自己的代码。C 是 Integration Owner，不是“一个人包办所有 QA”。

四个必须守住的内部节点

10/13
Core Code Deadline
核心模块 + ⩾20 tests

10/16
Integration Deadline
交叉测试 + 主要 bug 修复

10/22
Documentation Deadline
README + Report 完成

10/25 22:00
Internal Final Deadline
次日只负责提交，不开发

> **最重要的一条：**不要等到 10 月 25 日才第一次把三个成员的代码拼起来。真正决定项目是否顺利的，是 10 月 6 日的 API 统一和 10 月 16 日前的集成。

Prepared for NumCompute Assignment 2.1 • Team of 3 • Planning horizon: 5 Oct 2026 - 26 Oct 2026